Lecture 5 : General Problem Solving - 2

V. Susheela Devi

NPTEL

Real world problems

- **Airline travel problem**
  - **States** : Each state obviously includes a location (e.g. an airport) and the current time. Because the cost of an action (a flight segment) may depend on previous segments, their fare bases and their status as domestic or international, the state must record extra information about these “historical” aspects
  - **Initial State** : This is specified by the user’s query
  - **Actions** : Take any flight from the current location, in any seat class, leaving after the current time, leaving enough time for within-airport transfer if needed
  - **Transition model** : The state resulting from taking a flight will have the flight’s destination as the current location and the flight’s arrival time as the current time
  - **Goal test** : Are we at the final destination specified by the user?
  - **Path cost** : This depends on monetary cost, waiting time, flight time, customs and immigration procedures, seat quality, time of day, type of airplane, frequent-flyer mileage awards, and so on
- There are additional complications to handle the byzantine fare structures that airlines impose
- A really good system should include contingency plans – such as backup reservation on alternate flights – to the extent that these are justified by the cost and likelihood of failure of the original plan

NPTEL

Real world problems (contd.)

- **Touring problems :**
  - These are closely related to route-finding problems, but with an important difference
  - E.g. “Visit every city in Figure 3.2 at least once, starting and ending in Bucharest”
  - As with route finding, the actions correspond to trips between adjacent cities
  - The state space, however, is quite different
  - Each state must include not just the current location but also the set of cities the agent has visited
  - So the initial state will be : *In(Bucharest),Visited({Bucharest}),* a typical intermediate state would be

    In(Vaslui),Visited({Bucharest,Urzicene,Vaslui})
  - The goal test would check whether the agent is in Bucharest and all 20 cities have been visited

NPTEL

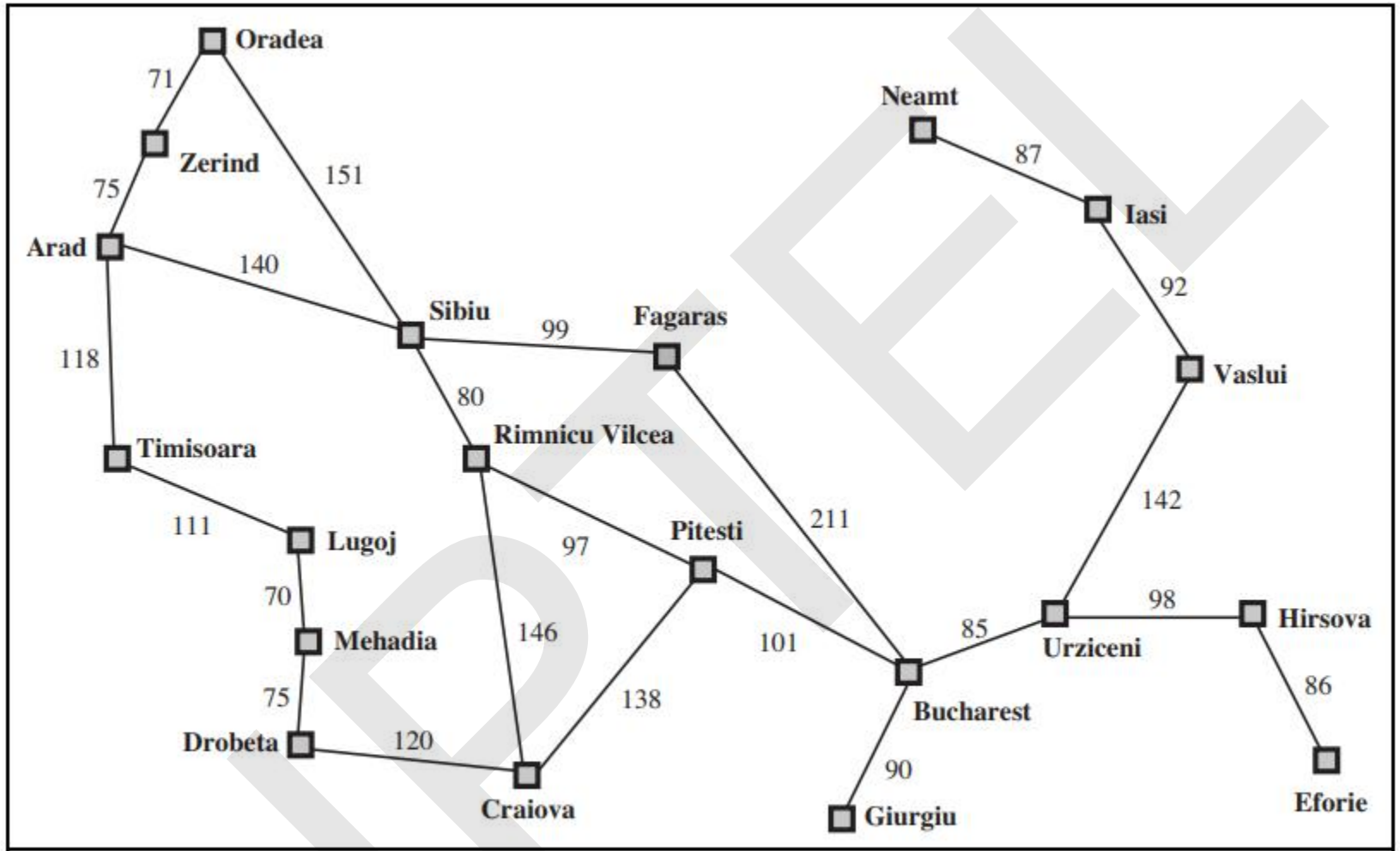


Figure 3.2 A simplified road map of part of Romania.

From Russel and Norvig, “Artificial Intelligence : A Modern Approach”

NPTEL

Real world problems (contd.)

- Traveling salesman problem (TSP)
  - It is a touring problem in which each city should be visited exactly once
  - The aim is to find the shortest route
  - The problem is known to be NP-hard, but an enormous amount of effort has been expended to improve the capabilities of TSP algorithms
  - In addition to planning trips for traveling salespersons, these algorithms have been used for tasks such as planning movements of automatic circuit-board drills and of stocking machines on shop floors

NPTEL

Real world problems (contd.)

- **Automatic assembly sequencing**
  - Automatic assembly sequencing of complex objects by a robot was first demonstrated by FREDDY (Michie 1972)
  - At present, the assembly of intricate objects such as electric motors is economically feasible
  - In assembly problems, the aim is to find an order in which to assemble the parts of some object
  - If the wrong order is chosen, there will be no way to add some part later in the sequence without undoing some of the work already done
  - Checking a step in the sequence for feasibility is a difficult geometrical search problem closely related to robot navigation
  - Thus, the generation of legal actions is the expensive part of assembly sequencing
  - Any practical algorithm must avoid exploring all but a tiny fraction of the state space
  - Another important assembly problem is **protein design**, in which the goal is to find a sequence of amino acids that will fold into a three-dimensional protein with the right properties to cure some disease

NPTEL

Searching for solutions

- Having formulated some problems, we need to now solve them
- A solution is an action sequence, so search algorithms work by considering various possible action sequences
- The possible action sequences starting at the initial state form a **search tree** with the initial state at the root
- The branches are actions and the **nodes** correspond to states in the state space of the problem
- Figure 3.6 shows the first few steps in growing the search tree for finding a route from Arad to Bucharest
- The root node of the tree corresponds to the initial state, *In(Arad)*
- The first step is to test whether this is a goal state
- Then we need to consider taking various actions
- We do this by **expanding** the current state; i.e. applying each legal action to the current state, thereby **generating** a new set of states
- In this case, we add three branches from the **parent node** *In(Arad)* leading to three new **child nodes** : *In(Sibiu), In(Timisoara) and In(Zerind)*
- Now we must choose which of these three possibilities to consider further

NPTEL

Searching for solutions (contd.)

- This is the essence of search – following up one option now and putting the others aside for later, in case the first choice does not lead to a solution
- If we choose Sibiu first, we check to see if it is a goal state (it is not) and then we expand it to get *In(Arad),In(Fagaras),*and *In(RimnicuVilcea)*
- We can then choose any of these four or go back and choose Timisoara or Zerind
- Each of these six nodes is a **leaf node**, that is, a node with no children in the tree
- The set of all leaf nodes available for expansion at any given point is called the **frontier**
- In Figure 3.6, the frontier of each tree consists of those nodes with bold outlines
- The process of expanding nodes on the frontier continues until either a solution is found or there are no more states to expand
- Figure 3.6 gives the general TREE-SEARCH algorithm
- All search algorithms share this basic structure; they vary primary according to how they choose which state to expand next which is called the **search strategy**

NPTEL

Searching for solutions (contd.)

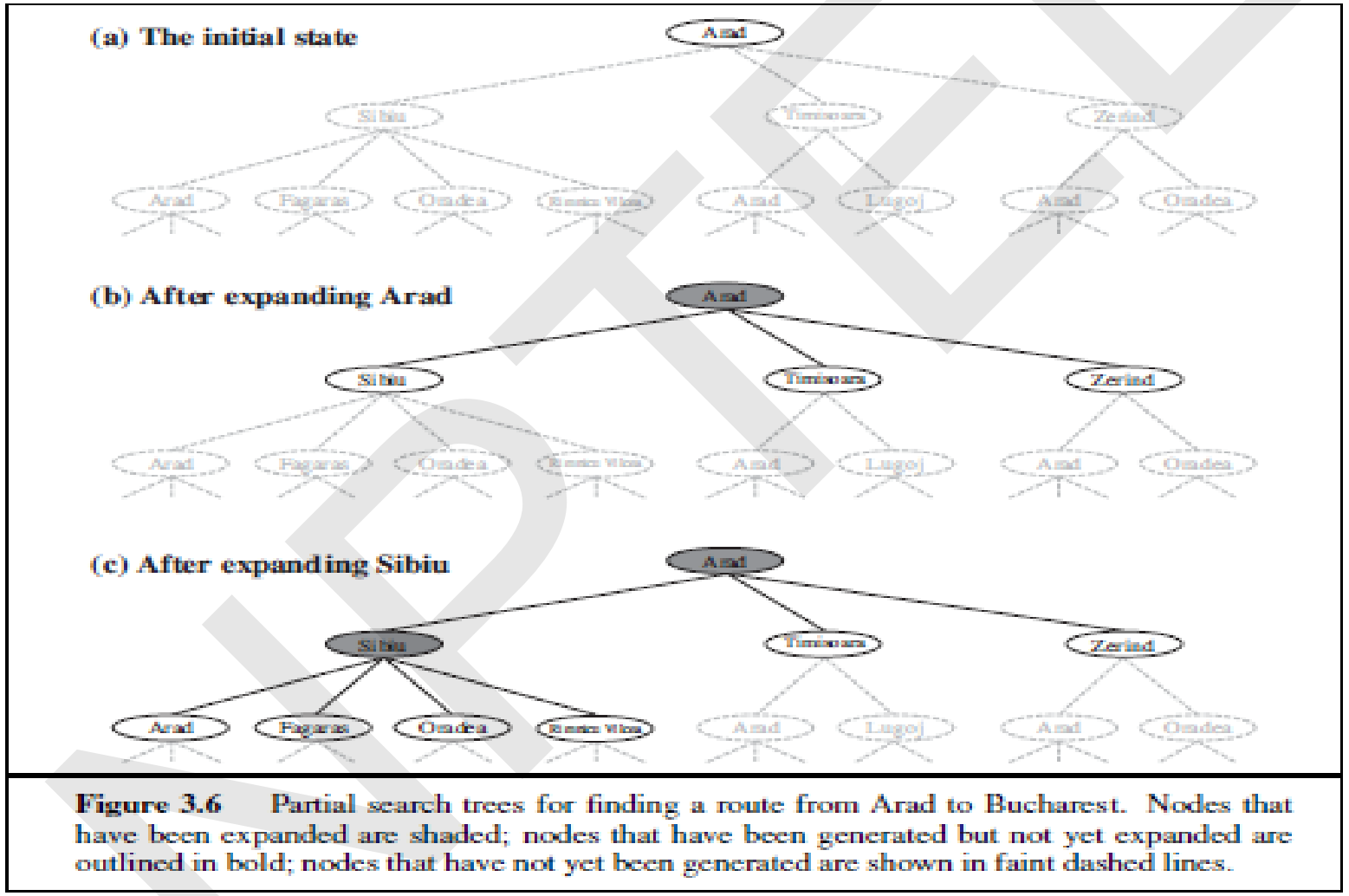


Figure 3.6 Partial search trees for finding a route from Arad to Bucharest. Nodes that have been expanded are shaded; nodes that have been generated but not yet expanded are outlined in bold; nodes that have not yet been generated are shown in faint dashed lines.

From Russel and Norvig, "Artificial Intelligence : A Modern Approach"

Searching for solutions (contd.)

- If we look at the search tree in Figure 3.6, it includes the path from Arad to Sibiu and back to Arad again
- *In(Arad)* is a repeated state in the search tree, generated in this case by a loopy path
- Considering such loopy paths means that the complete search tree for Romania is infinite because there is no limit to how often one can traverse a loop
- On the other hand, the state space has only 20 states
- Loops can make certain algorithms to fail, making otherwise solvable problems unsolvable
- So there is no need to consider loopy paths
- Since path costs are additive and step costs are nonnegative, a loopy path to any given state is never better than the same path with the loop removed
- We can also have redundant paths which exist whenever there is more than one way to get from one state to another
- We can have Arad-Sibiu (140km long) and Arad-Zerind-Oradea-Sibiu (297 km long). The second path is redundant
- To reach the goal, there is no reason to keep more than one path to any given state, because any goal state that is reachable by extending one path is also reachable by extending the other

NPTEL

Searching for solutions (contd.)

- Redundant paths are unavoidable and include all problems where the actions are reversible, such as route-finding and sliding block puzzles
- Route finding on a rectangular grid (Fig. 3.9) is used a lot in computer games. Each state has four successors, so a search tree of depth d that has repeated states has 4^d leaves but only $2d^2$ distinct states within d steps of any given state. For d=20, this means about a trillion nodes but only about 800 distinct states
- Following redundant paths can cause a tractable problem to become intractable
- This is true even for algorithms that know how to avoid infinite loops
- The way to avoid exploring redundant paths is to remember where one has been
- To do this, we augment the TREE-SEARCH algorithm with a data structure called the **explored set** , which remembers every expanded node
- Newly generated nodes that match previously generated nodes – ones in the explored set or the frontier – can be discarded instead of being added to the frontier
- The new algorithm, called GRAPH-SEARCH, is shown in Figure 3.7

NPTEL

Searching for solutions (contd.)

- The search tree constructed by GRAPH-SEARCH algorithm contains at most one copy of each state, so we can think of it as growing a tree directly on the state-space graph, as shown in Figure 3.8
- The frontier **separates** the state-space into the explored region and the unexplored region, so that every path from the initial state to an unexplored state has to pass through a state in the frontier
- This property is illustrated in Figure 3.9
- As every step moves a state from the frontier into the explored region while moving some states from the unexplored region into the frontier, we see that the algorithm is systematically examining the states in the state space, one by one, until it finds a solution

NPTEL

Searching for solutions (contd.)

```
function TREE-SEARCH( problem) returns a solution, or failure
    initialize the frontier using the initial state of problem
    loop do
        if the frontier is empty then return failure
        choose a leaf node and remove it from the frontier
        if the node contains a goal state then return the corresponding solution
        expand the chosen node, adding the resulting nodes to the frontier
```

```
function GRAPH-SEARCH( problem) returns a solution, or failure
    initialize the frontier using the initial state of problem
    initialize the explored set to be empty
    loop do
        if the frontier is empty then return failure
        choose a leaf node and remove it from the frontier
        if the node contains a goal state then return the corresponding solution
        add the node to the explored set
        expand the chosen node, adding the resulting nodes to the frontier
            only if not in the frontier or explored set
```

Figure 3.7 An informal description of the general tree-search and graph-search algorithms. The parts of GRAPH-SEARCH marked in bold italic are the additions needed to handle repeated states.

From Russel and Norvig, "Artificial Intelligence : A Modern Approach"

Searching for solutions (contd.)

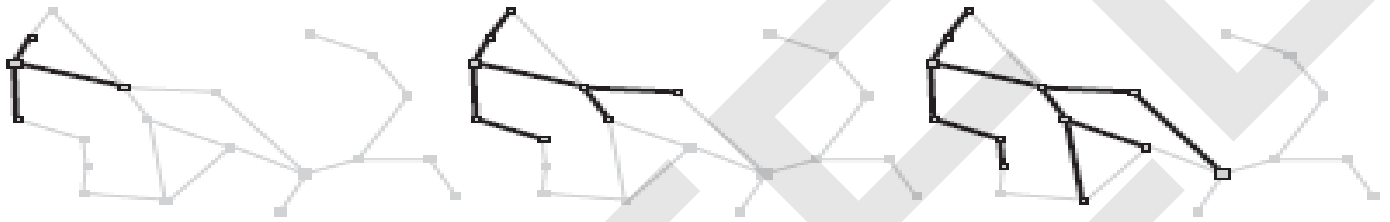


Figure 3.8 A sequence of search trees generated by a graph search on the Romania problem of Figure 3.2. At each stage, we have extended each path by one step. Notice that at the third stage, the northernmost city (Oradea) has become a dead end: both of its successors are already explored via other paths.

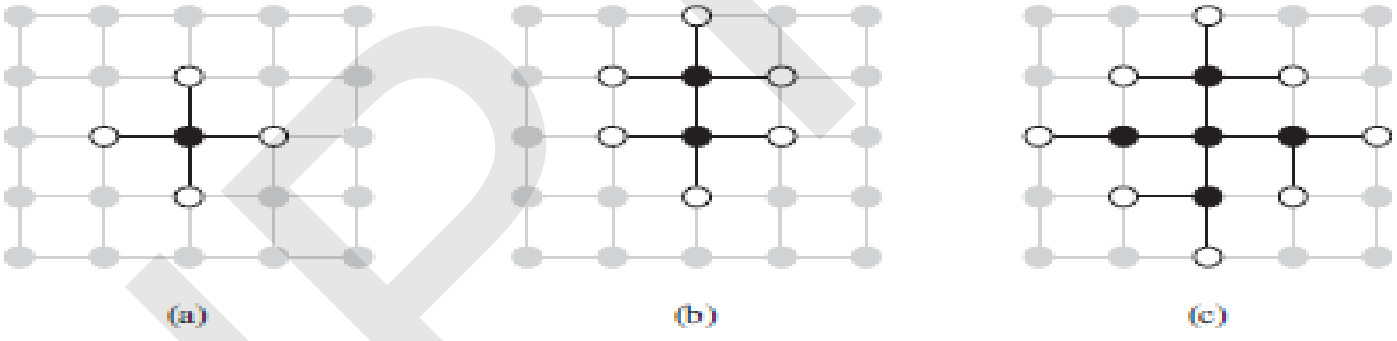
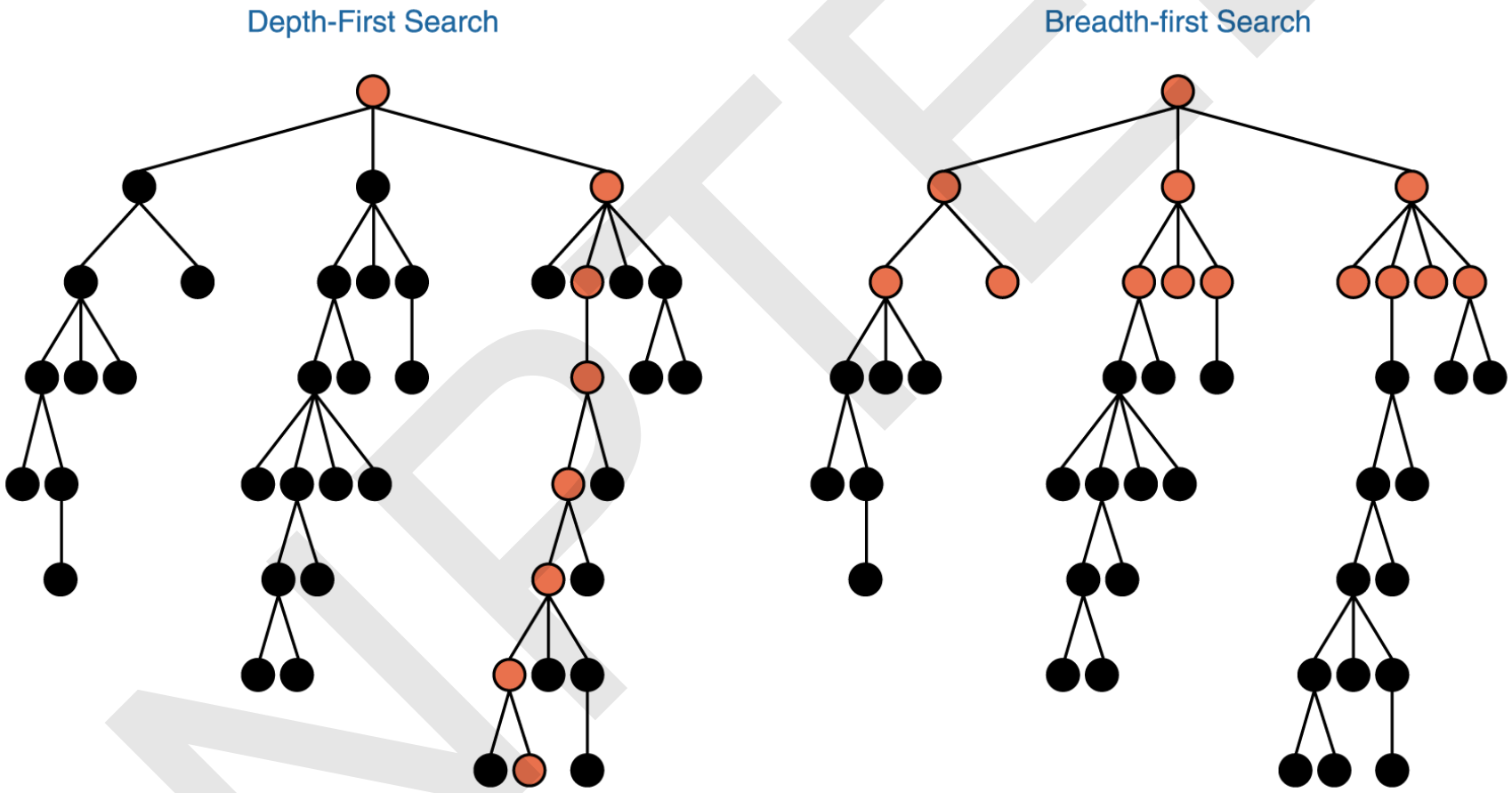


Figure 3.9 The separation property of GRAPH-SEARCH, illustrated on a rectangular-grid problem. The frontier (white nodes) always separates the explored region of the state space (black nodes) from the unexplored region (gray nodes). In (a), just the root has been expanded. In (b), one leaf node has been expanded. In (c), the remaining successors of the root have been expanded in clockwise order.

From Russel and Norvig, “Artificial Intelligence : A Modern Approach”

Search Trees



NPTEL

Search Strategies

- Uninformed or blind search
- Informed or heuristics search
- Game playing search
- Randomized search algorithms
  - Hill climbing
  - Stochastic search like genetic algorithms

NPTEL

Performance of problem solving

- Four criteria to be used
  - **Completeness :** Is the algorithm guaranteed to find a solution when there is one?
  - **Optimality :** Does the strategy find the optimal solution?
  - **Time complexity :** How long does it take to find a solution?
  - **Space complexity :** How much memory is needed to perform the search?
- Time and space complexity are always considered with respect to some measure of the problem difficulty
- In theoretical CS, the typical measure is the size of the state space graph, $|V| + |E|$, where V is the set of vertices (nodes) of the graph and E is the set of edges (links)
- In AI, the graph is often represented implicitly by the initial state, actions and transition model and is frequently infinite

NPTEL

Performance (contd.)

- Complexity is therefore expressed in terms of b, the branching factor or maximum number of successors of any node; d, the depth of the shallowest goal node (i.e. the number of steps along the path from the root); and m, the maximum length of any path in the state space
- Time is measured in terms of the number of nodes generated during search
- Space is measured in terms of the maximum number of nodes stored in memory
- To assess the effectiveness of a search algorithm, we can consider the search cost which depends on the time and space complexity
- We can also assess effectiveness by using the total cost, which combines search cost and the path cost of the solution found
- For the problem of finding a route from Arad to Bucharest, the search cost is the amount of time taken by the search and the solution cost is the total length of the path in kilometers

NPTEL